

NODE.JS FUNDAMENTALS

1. What is Node.js?

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser.

It is commonly used for building backend applications and APIs.

Node.js uses Google's V8 JavaScript engine.

2. HTTP Server

Node.js provides the HTTP module for creating web servers.

Example:

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.end("Hello from Node.js");

});

server.listen(3000);
```

The server listens for incoming HTTP requests.

3. Express.js

Express.js is a web framework built on top of Node.js.

It makes it easier to create:

- APIs
- Routes
- Middleware
- Controllers
- Web servers

Example:

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.json({
    message: "Hello"
  });
});
```

```
app.listen(3000);
```

4. Routes

A route defines how the server responds to a particular HTTP request.

Example:

```
app.get("/users", (req, res) => {  
  res.json({  
    users: []  
  });  
});
```

This route handles a GET request to /users.

5. HTTP Methods

Common HTTP methods include:

GET

Used to retrieve data.

POST

Used to create data.

PUT

Used to replace existing data.

PATCH

Used to partially update data.

DELETE

Used to delete data.

6. Middleware

Middleware is a function that runs during the request-response cycle.

Middleware can:

- Modify the request
- Modify the response
- Perform authentication
- Log requests
- Validate data

Example:

```
app.use((req, res, next) => {  
  
  console.log(req.method);  
  
  next();  
  
});
```

The next() function passes control to the next middleware or route handler.

7. REST API

A REST API provides endpoints through which clients can communicate with a backend server.

Example:

```
GET /users  
POST /users  
GET /users/10  
PATCH /users/10  
DELETE /users/10
```

8. Request and Response

The request contains information sent by the client.

The response contains information sent back by the server.

Example:

```
app.get("/users", (req, res) => {  
  
  res.status(200).json({  
    message: "Users fetched successfully"  
  });  
  
});
```

9. Status Codes

Common HTTP status codes include:

```
200 - Request successful  
201 - Resource created  
400 - Bad request  
401 - Unauthorized  
403 - Forbidden  
404 - Resource not found  
500 - Internal server error
```

10. Async Programming

Node.js commonly uses asynchronous programming.

Promises and `async/await` are frequently used when working with databases and APIs.

Example:

```
async function getUsers() {  
    const users = await User.find();  
    return users;  
}
```

11. Environment Variables

Sensitive configuration such as API keys and database URLs should not be hardcoded in application code.

Environment variables can be stored in a `.env` file.

Example:

```
PORT=3000  
DATABASE_URL=your_database_url
```

12. Summary

Node.js is commonly used for backend development.

Express provides:

- Routing
- Middleware
- Request handling
- Response handling
- REST API development